



Computer Graphics

Dingzhuanlian

Anhui University

Chap. 2 Foundation

- **Basics of Computer Graphics**
 - **Foundation of Mathematics** 附录A
 - **Color Model** 颜色模型
 - **Transformation** 变换相关的概念
 - **OpenGL**图形开发工具

Foundation of mathematics

- 矢量运算
- 矩阵运算
- 线性方程组的求解
- 齐次坐标 Homogeneous Coordinate

Homogeneous Coordinate

- **齐次坐标**表示就是用 $n+1$ 维向量表示一个 n 维向量。

$$(x, y) \Leftarrow (xh, yh, h) \quad h \neq 0$$

- 普通坐标与齐次坐标的关系为“**一对多**”。

- **规范化齐次坐标**表示就是 **$h=1$** 的齐次坐标表示。

$$(x, y) \Leftarrow (x, y, 1)$$

- 规范化齐次坐标中，前 n 个坐标就是普通坐标系下的 n 维坐标。用**规范化**齐次坐标来保证**唯一性**。

- **对于图形来说，没有实质性的差别，但是却给后面矩阵运算提供了可行性和方便性。有了齐次坐标,变换过程可以统一写成矩阵乘法形式。**

Homogeneous Coordinate

- **Homogeneous coordinate is a unified way to represent all transformations.** 图形通常要进行一系列变换, 希望能够把变换统一表示为矩阵的乘法形式。

- Take 2D affine transformation as an example:

$$x' = ax + by + c$$

$$y' = dx + ey + f$$

- Represent it in vector-matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ d & e \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} c \\ f \end{bmatrix}$$

$$p' = M p + t$$

Homogeneous Coordinate

- The objective of HC is to use a 3D column matrices to represent both points and vectors in 2D.
- Add an extra dimension

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ d & e \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} c \\ f \end{pmatrix}$$

$$p' = M p + t$$

Formulation in 2D

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$p' = M p$$

Homogeneous formulation (合并矩阵运算中的乘法和加法，统一表示为矩阵的乘法形式)

Homogeneous Coordinate

- Add an extra dimension
 - in 2D, we use 3 x 3 matrices
 - In 3D, we use 4 x 4 matrices
(引入新的维度后, 使用4*4的作用矩阵, 同时使用 (x,y,z,w) 来表示一个点或向量)
- Each point has an extra value, w

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}$$
$$p' = \textcolor{red}{M} p$$

Homogeneous Coordinate

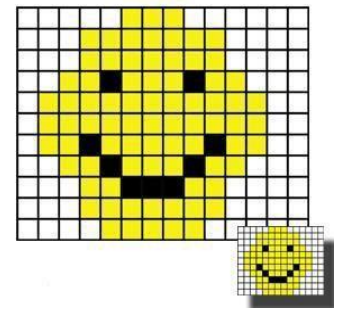
- Most of the time $w = 1$, and we can ignore it

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

- 当齐次坐标被仿射矩阵作用时，仿射变换矩阵最后一行一定是0 0 0 1， $w'=w$ ，变换前的 w 与变换后的 w 完全一样，变回真实普通坐标时，所有点都除以同一个 w ，没有远近差别，没有近大远小；
- 投影变换矩阵（特别是透视投影），每个点的 w' 都不同，变回真实坐标时，远处的点， w' 大（除以一个的大数），变小；近处的点， w' 小（除以一个的小数），变大。出现近大远小，透视效果。

Color Model 颜色模型

Image & Pixel

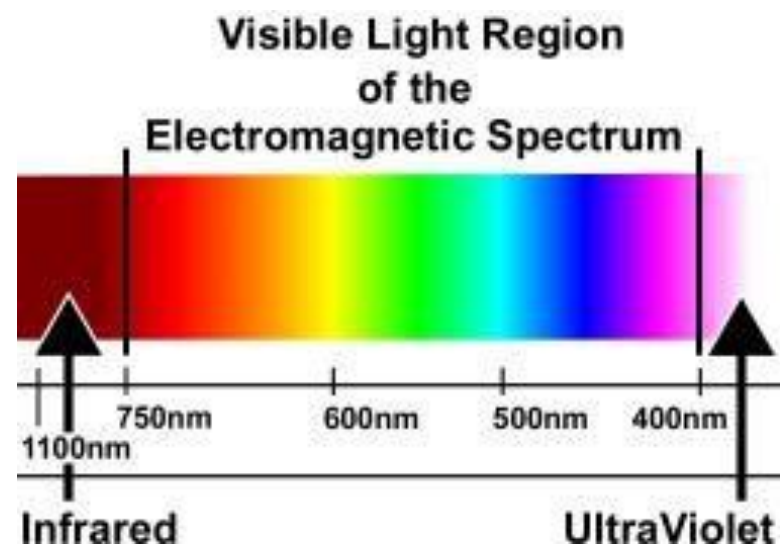


- 图形学的一个主要目的就是给我一个问题，我要**确定这个图形上每个像素点的值**。比如给你一个视点，一个投影平面，然后有一个场景，我要把它绘制出来，无非就是把投影平面上**每个像素的值**给它算出来，然后把它显示到屏幕上。
 - Image、Pixel
 - image could be treated as a 2-dimensional discrete function $f(x,y)$.
图像由像素组成，函数 f 的定义域是由矩阵排列着的许多格子组成。
 - each discrete grid is called pixel (**像素**)
- Usually, each pixel has a RGB value for color image or a scalar value for gray image

Color Perception 色彩感知 (颜色视觉)

- **What is Color?**

- Colors are the sensations(感知) that arise from light energy of different wavelengths.
 - electromagnetic waves (电磁波) with different wavelengths are corresponding to different colors
 - Human's eye are sensitive for electromagnetic wave of wavelengths between 380 to 760 nm



Spectral Distribution of Light 光的谱分布

—Light is a mixture of many wavelengths, each with some intensity

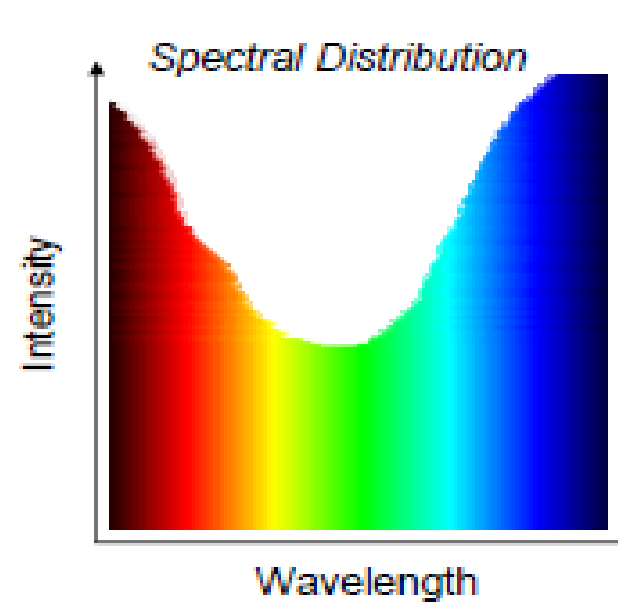
不同波长的电磁波按照某种能量分布混合叠加而成

E.g., White Light means light include all wavelengths with the same intensity. 由所有可见波长的电磁波以相等的强度混合得到.

- 如何刻画某一种光, 如何辨别某种颜色呢? --谱分布.

—*spectral distribution*: intensity as a function of wavelength over the entire spectrum.

光在各个可见波长的分量的强度分布函数称为光的谱分布.

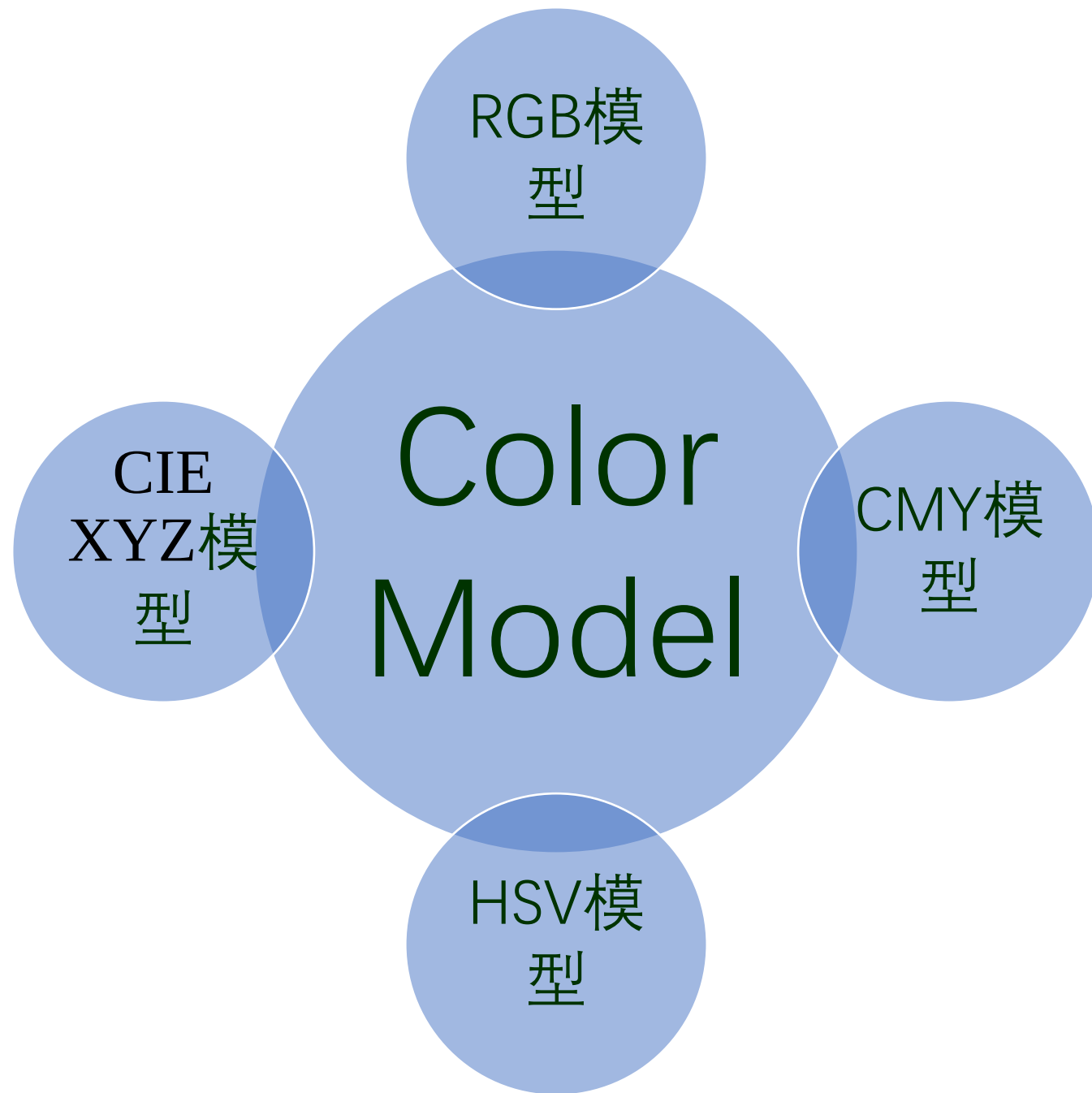


↓
Sample Color



(这种光在每个波长上它能量有多大, 这样就得到一条曲线, 这条曲线我们称为谱分布)

- So, colors can be represented as such distribution function. (与光类似, 颜色也可以使用谱分布函数来进行描述)
- However, use spectrum distribution for color representation is too complicated, and the corresponds is multiple-to-one.
- There exists a situation as — different spectrum distribution with the same color. (异谱同色)



RGB Color Space

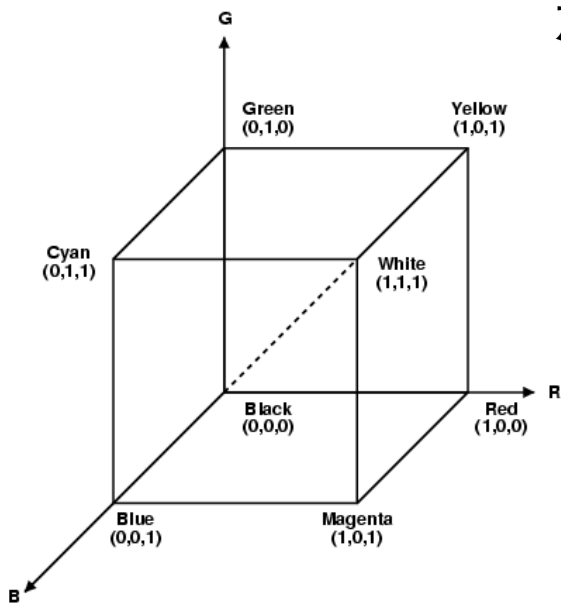
- We first introduce color space for color representation. RGB is commonly used color space in Graphics
 - Color is represented as RGB triple (r,g,b);
 - People perform almost all operations on each channel separately
 - usually, each color value is a float value range from 0 to 1, or 0-255 if we use 8 bit integers.
通常可以将r,g,b分别规整为[0,1]内的浮点数；当使用8bit进行存储时，r,g,b通常取值为[0,255]内的整数。

RGB Color Space

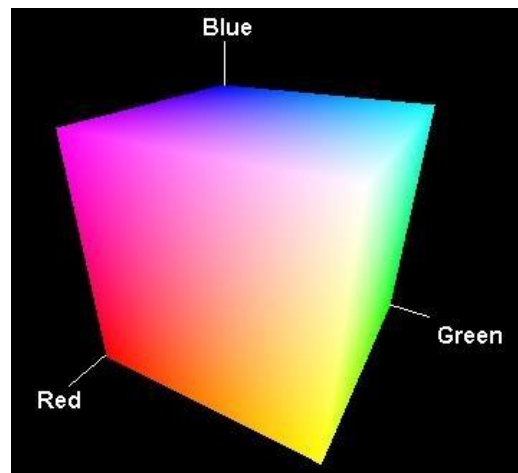
- Color are represented as combinations of three primaries: red, green, blue三个基本颜色
 - $\mathbf{C} = r\mathbf{R} + g\mathbf{G} + b\mathbf{B}$
 - The reason why we pick red, green, and blue?
 - essentially because of the structure of our visual system 以人类视觉的三刺激理论为基础
 - Our visual system is sensitive to those three colors
人眼的视网膜中有三种锥状视觉细胞，分别对红、绿、蓝三种光最敏感

RGB Color Space

- Color are represented as combinations of three primaries:
red, **green**, **blue**
 - $C = rR + gG + bB$

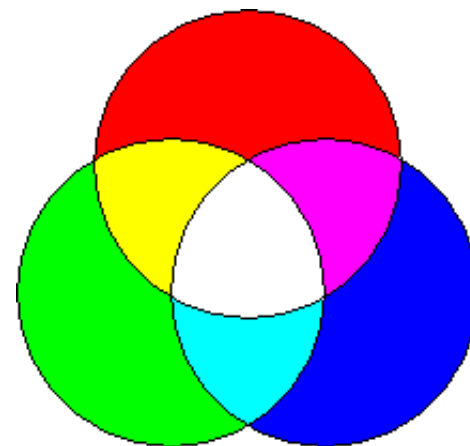


加色系统，加色三原色 (0, 0, 0)为黑，(1, 1, 1)为白



颜色立方体

该立方体中每个点都有个RGB值，同时也对应一种颜色

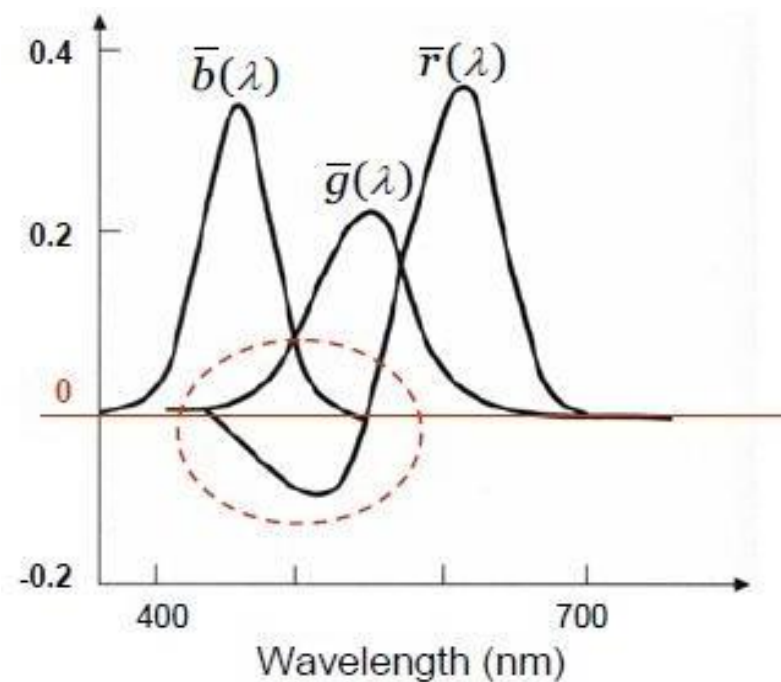


颜色叠加

颜色的叠加会出现其他效果

RGB Color Space

- Unfortunately, Some colors cannot be written as combinations of RGB triples, because some parts of the red curve is negative



图：所有可见光波长的RGB分量图

当波长为500时，这个波长的光如何表示？
红色光取值的百分比是负值，这样就出现问题了

Other Color Spaces

- So some other color space are also used
 - CMY
 - HSV
 - CIE XYZ

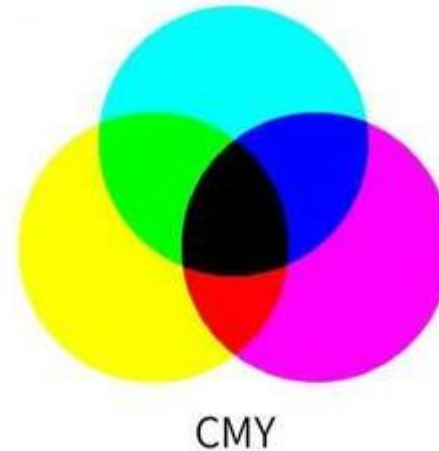
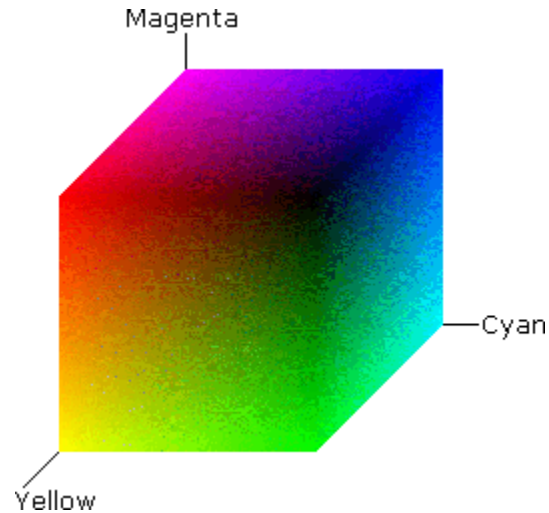
CMY

- CMY :The other set of primaries besides RGB
 - Cyan(青), magenta (品红) , and yellow — complements (补色) of red, green, blue

$$\begin{array}{c} \text{C} \\ \text{M} \\ \text{Y} \end{array} = \begin{array}{c} 1 \\ 1 \\ 1 \end{array} - \begin{array}{c} \text{R} \\ \text{G} \\ \text{B} \end{array}$$

- It's called **subtractive** primaries (减色系统, 减色原色, 减法三原色)
 - RGB are additive primaries(加色原色), start with black, add up to white
加色系统, (0, 0, 0)为黑, (1, 1, 1)为白
 - CMY, start with white and subtract colors from white
(0, 0, 0)为白, (1, 1, 1)为黑

CMY Color Cube



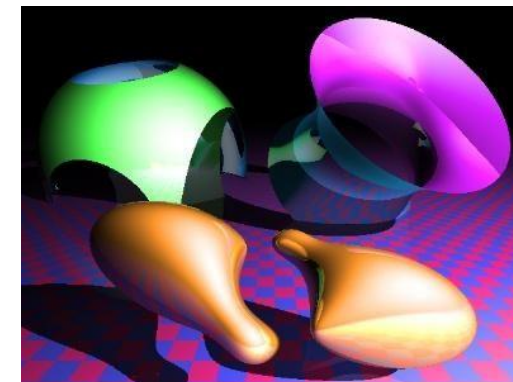
RGB and CMY

$$[C \ M \ Y] = [1 \ 1 \ 1] - [R \ G \ B] ; \quad [R \ G \ B] = [1 \ 1 \ 1] - [C \ M \ Y]$$

- RGB是以黑色介质为基础，通过光源三原色（红、绿、蓝）不同比例的亮度叠加，来获得各种不同的颜色。只要输出端是电子屏幕，那么就需要使用RGB模式；
- CMY是以白色介质为基础，通过印刷三原色（青、品红、黄）不同比例的油墨混合，吸收原始色光中的对应波长，从而获得各种不同颜色的反射效果。主要应用于印刷出版行业。印刷采用色料三原色加黑色的四色模式CMYK，实现全彩印刷。

HSV Color space

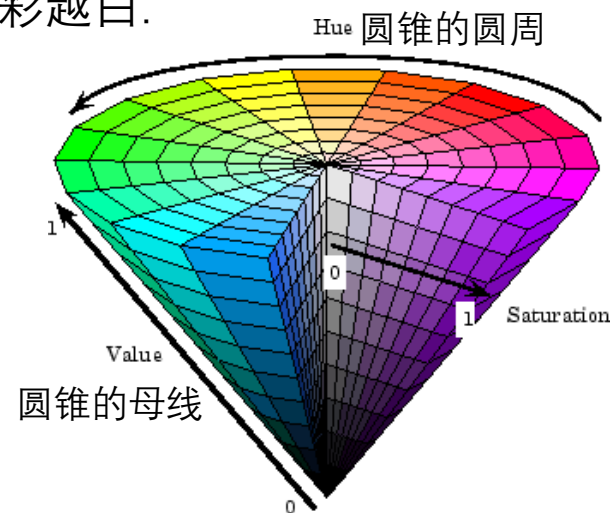
- True color pictures with 3 components (RGB), each has 256 possible value, so we have up to 16,777,216 possible colors.
- Controlling this huge amount of possibilities is almost impossible without the HSV space.
使用 RGB 颜色空间来描述和定位如此大量的不同颜色非常困难
- HSV provides an intuitive method for color selection.
- Applications: arts, image processing, fractal pictures分形图像...(如美颜, 调色, 图像分割)



◆ RGB是给机器看的，HSV是给人看的；RGB机器天生就用它进行屏幕显示、图像存储、计算机图形学渲染等，机器读起来最快最直接，但对人不友好，如你想更红一点，RGB三个值都要改。对HSV而言，人理解颜色，算法处理颜色都用它。如颜色检测、图像分割、美颜、调色、修图。

HSV Color space

- HSV: a color space in cone-shaped 圆锥形的颜色空间
 - Hue (色调) means the base color, which is a main factor indicates difference between colors.它描述了色彩的本征属性，是指能够比较确切地表示某种颜色色别的名称，如红色、黄色等。
 - Saturation(饱和度) is for purity of color色彩的鲜艳程度或纯度 (decrease= adding white)饱和度越低 色彩越白.
 - Value of brightness(亮度),色彩的明亮程度 (decrease=adding black)亮度越低，色彩越黑
- ✓ more user-friendly than RGB



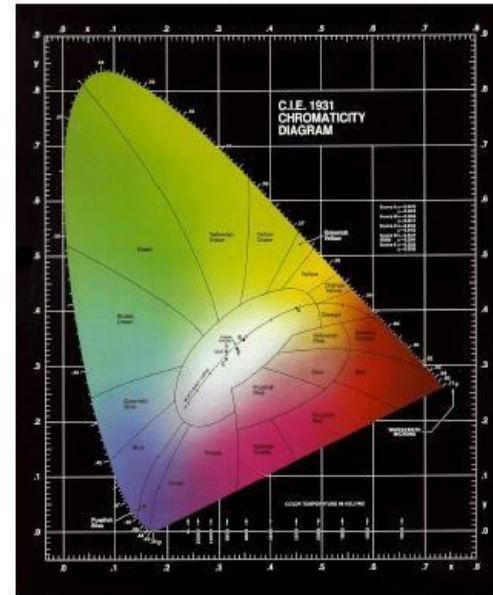
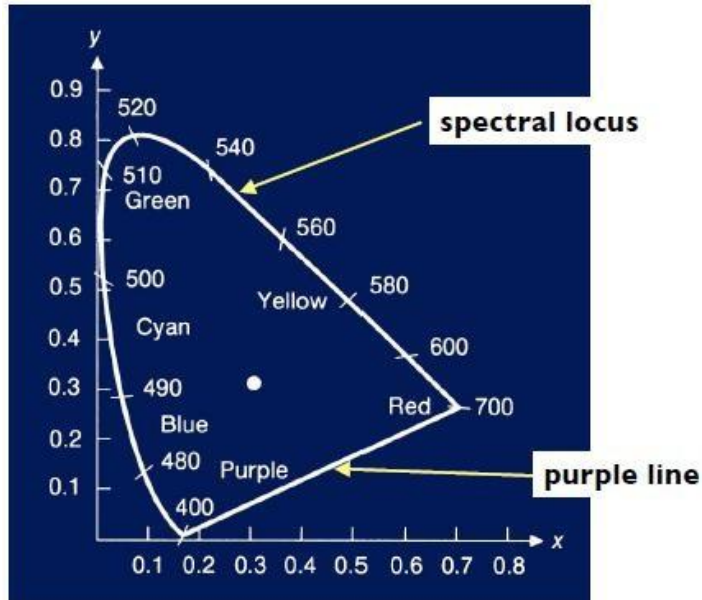
CIE XYZ

- CIE XYZ color space
 - Proposed by CIE (International Commission on Illumination国际发光照明委员会) in 1931
- It can represent all perceptible colors.
 - 可以表示所有可感知的颜色(while RGB not)
 - most used in color science更多应用于颜色科学的研究
 - based on **human perception studies**与人类的感知系统密切相关

CIE XYZ (Chromaticity Diagram色度图)

This diagram shows how to use XYZ to represent colors.

- ✓ 假设了并不存在于自然界的三种原色，即**理论三原色XYZ**，相应的形成了XYZ颜色空间。XYZ三刺激值是由**RGB彩色空间线性变换转换得到**，变换后的空间就是CIE XYZ彩色空间，相当于使用匹配颜色的XYZ基底来代替RGB基底来表示颜色。
- ✓ 如图所示，马蹄色度图画出人眼能感知的所有颜色。马蹄形外轮廓是**纯光谱色**(只有单一波长的光)，饱和度最高，颜色最纯，马蹄内部点是边缘纯色的**混合色**，混合比例决定了位置，越靠近中心越白，饱和度越低。光谱的红色波段集中在图的右下部，绿色波段集中在图的上部，蓝色波段集中在轨迹图的左下部。



图形系统中几个名词概念Some Concepts

- **刷新频率 Refresh rate**
 - 在屏幕上重复画图的频率
- **像素Pixel—每个屏幕上的点**
- **帧缓存—frame buffer**
 - 显示屏幕图像所需要的存储空间
 - 帧frame——整个屏幕范围
- **点距 spot distance**
 - 指荧光屏上两个同样颜色荧光点之间的距离
- **分辨率resolution**
 - 屏幕图像的密度。一般表示为水平线上面的点数乘以垂直线上面的点数

颜色相关的一些名词概念Some Concepts

- 颜色的灰度值 Gray level
- 单色 1bit
- 256色 8bit
- 65536 16bit
- 真彩色 24bit
- 问题：真彩色图像在1024*768的屏幕分辨率下帧缓存需要多大？

$$1024 \times 768 \times 24 / 8 = 2359296 \text{ B} = 2.25\text{M}$$

Transformation

Transformation 变换

- **Why Transformation**

- Computer graphics renders high-quality color images of a scene which is composed of 3D geometric models.

图形学关注于如何将由几何模型组成的三维场景绘制成高质量的彩色图像。

在图形学中，我们利用投影模拟眼睛的功能和效果，要将三维场景感知为二维图像。

- Transformations is extremely important in graphics, With them, you can position, reshape and animate objects, lights and cameras.

通过变换，可以简洁高效地设置和编辑三维场景，光照位置，以及视点方向。

Why transformation: an example

- Suppose we already have a procedure to draw a unit square $[0,1] \times [0,1]$
- If you want to write another procedure to draw a 2D rectangle with one corner at (lox, loy) and another diagonal corner at (hix, hiy) .

One Solution

- Rewrite the drawing rectangle procedure

```
drawRect(lox, loy, hix, hiy) {  
    glBegin(GL_QUADS) ; //绘制给出4个点的四边形  
    glVertex2f(lox, loy) ;  
    glVertex2f(hix, loy) ;  
    glVertex2f(hix, hiy) ;  
    glVertex2f(lox, hiy) ;  
    glEnd() ;  
}
```

- Drawing a rectangle could be done like this, how about drawing a complex model such as a teapot?

Solution using Transformation

```
drawRect(lox, loy, hix, hiy) {  
    glTranslate(lox, loy); //移动画布起点  
    glScale(hix-lox, hiy-loy); //拉伸画布  
    drawUnitSquare(0,0,1,1); //在拉伸过的画布上画一个单位正方形  
} //对当前的坐标系进行平移和缩放，然后在变换后的坐标系里画单位正方形，这个  
    单位正方形在原来的世界坐标系里就变成了想要的矩形。
```

This is a solution you should use in almost all graphics application for better speed, modularity, flexibility, ...这样的基于变换的解决方案在图形学中随时都需要用到；使用变换可以使代码更加快速、灵活、模块化。

What is a Transformation?

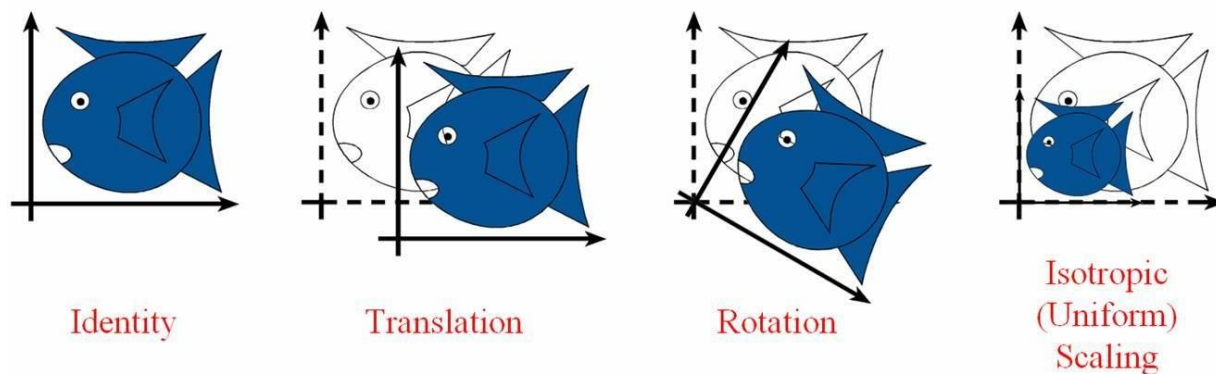
- A function that maps a point x to another point x' .
- Applications: Morphing (渐变变形, 变身), deformation (形状被改变, 扭曲), viewing, projection, real-time shadows, ...



Fig 1. Undeformed Plastic

Fig 2. Deformed Plastic

Simple Transformations



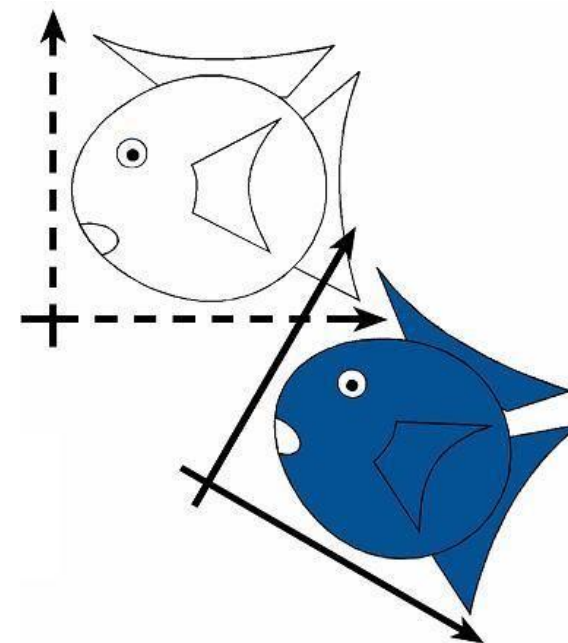
- Let's see some intuitive transformation from left to right:
 - **Identity(不变)** , **translation(平移)**, **rotation(旋转)** and **isotropic scaling(均衡缩放)**
- Transformations can be combined together
 - First rotate, then scale, then translate
- Transformations are invertible(可逆) operators

Transformation Classification(分类)

- Transformations could be classified into these categories:
 - Rigid-body Transformation (刚体变换)
 - Similarity Transformation(相似变换)
 - Linear Transformation(线性变换)
 - Affine Transformation(仿射变换)
 - Projective Transformation(投影变换)

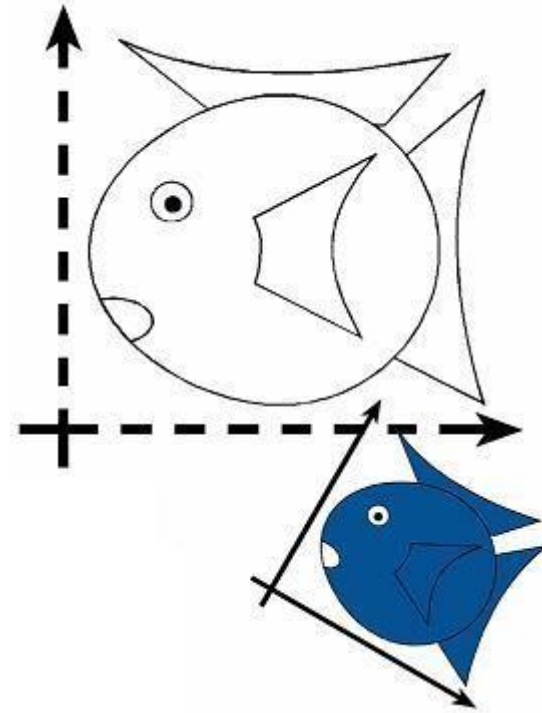
Rigid-body Transformation

- Preserve distance, angle and size
(保持长度, 角度, 大小不变)
- Include
 - Identity
 - translation
 - Rotation
 - and combinations of them
- **Scaling is not rigid body transformation**



Similarity Transformation

- Preserve angle
- Include
 - Identity
 - Translation
 - Rotation
 - Isotropic Scaling
- **not include general scaling**



Linear Transformation

- Preserve linearity of addition and scalar multiplication:

线性变换**要求**: 保持加法和数量乘法的线性

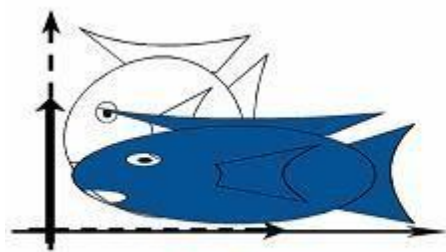
$$\mathbf{L}(\mathbf{p}+\mathbf{q}) = \mathbf{L}(\mathbf{p}) + \mathbf{L}(\mathbf{q})$$

向量相加再变换=分别变换再相加

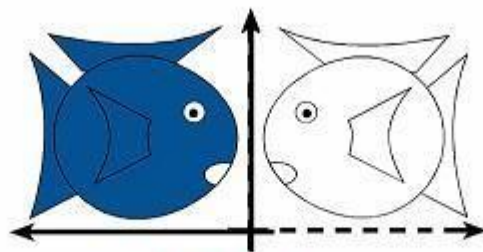
$$\mathbf{aL}(\mathbf{p}) = \mathbf{L}(\mathbf{ap})$$

变换再数乘=数乘再变换

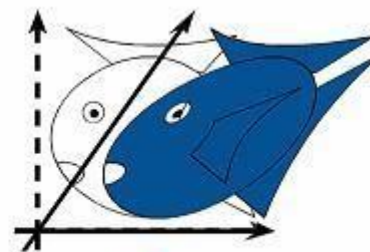
- Include
 - Identity, Rotation, Scaling
 - Reflection(对称), Shear(切变、剪切)
 - 错切变换是指图形沿某坐标方向产生不等量的移动而引起图形变形的一种变换。
 - But **translation is not linear** because of addition



Scaling



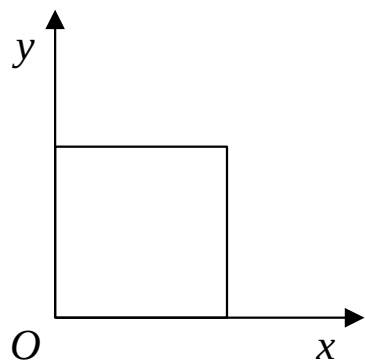
Reflection



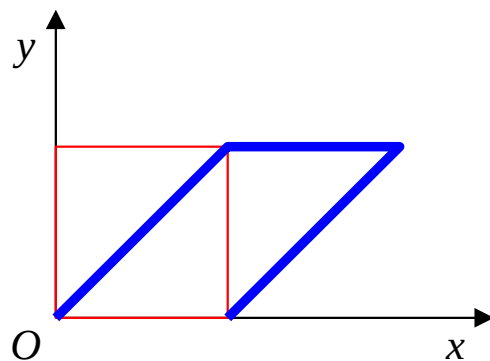
Shear

Shear 错切变换

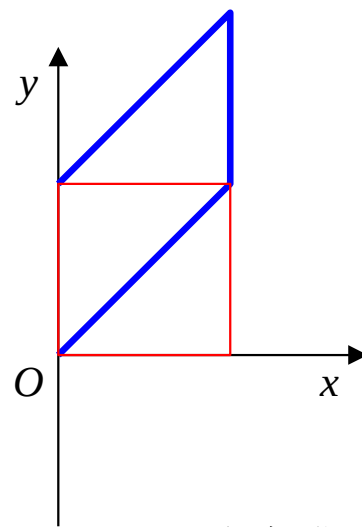
- 错切变换也称为剪切、错位变换，用于产生弹性物体的**变形处理**。



(a) 原图



(b) 沿x正方向错切

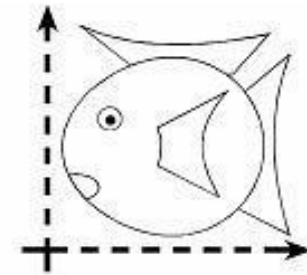
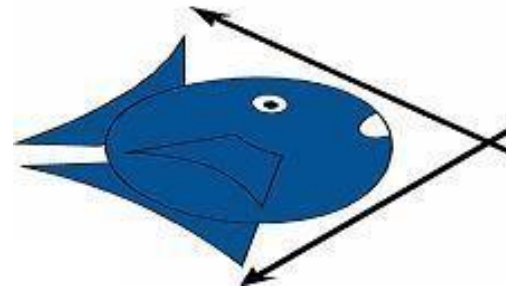


(c) 沿y正方向错切

图：错切变换

Affine Transformation

- Preserve **parallel** lines
 - If two lines are parallel before transformation, they are still parallel after transformation



- Include
 - linear transformation
 - similarity transformation

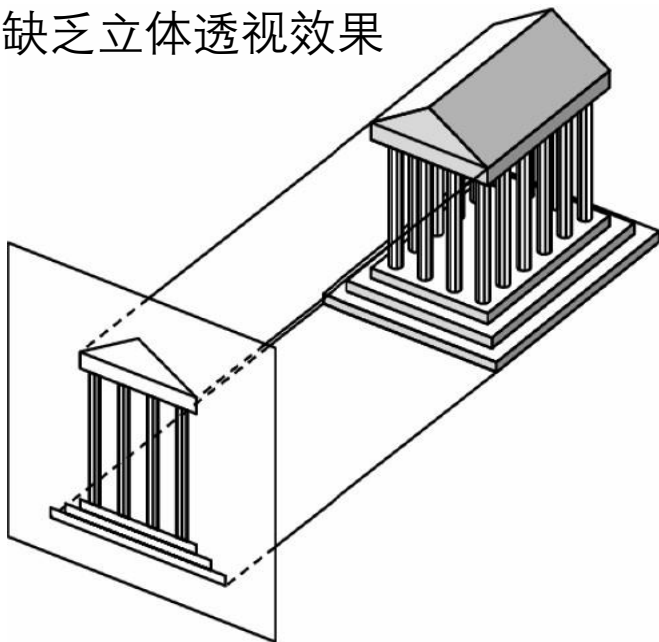
Projective Transformation

- **Our eyes could collapse 3D world to 2D image (brain then reconstructs 3D).** 我们的眼睛能将三维场景感知为二维图像，大脑则会将二维图像再重构回三维.
- **In computer graphics, we do it by projection.** 利用投影，模拟眼睛的功能和效果.
- **投影模式：** 是将 3D 变换成为 2D 的变换模式.
- **perspective/orthographic(透视投影/正交投影):** reduces 3D to 2D.

Projective Transformation

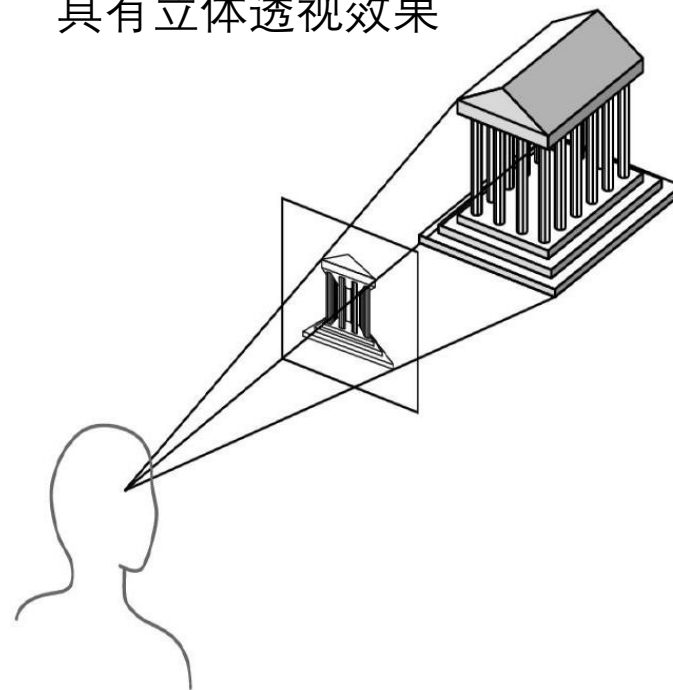
- 正交投影(Orthographic Projection)
- 透视投影(Perspective Projection)

- the focal point is at infinity, the rays are parallel and orthogonal to the image plane
- 缺乏立体透视效果

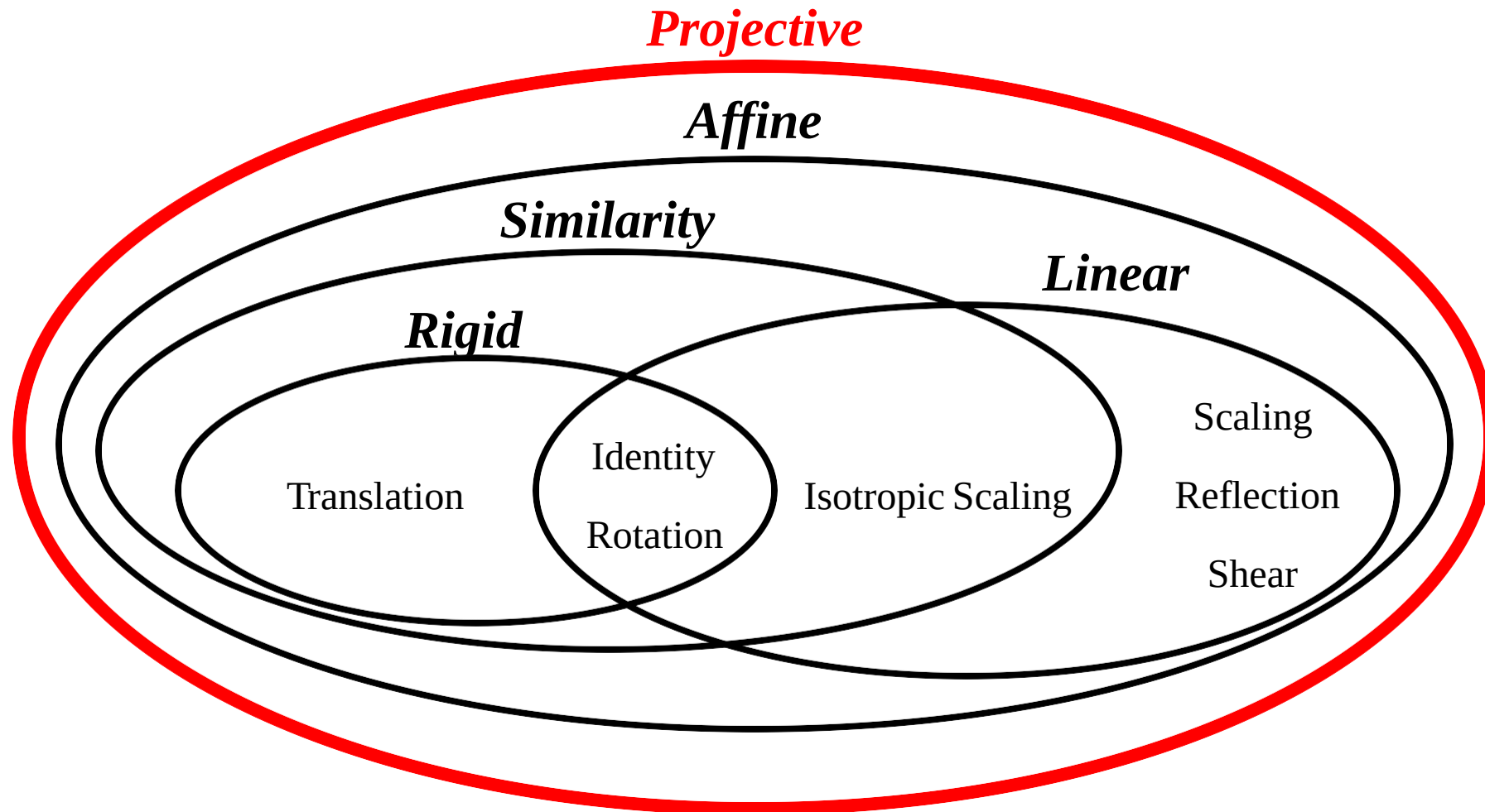


平行投影包含正交投影和斜投影。
投影线垂直于投影面称之为正交投影。

- Focus point is at finite distance
- 具有立体透视效果

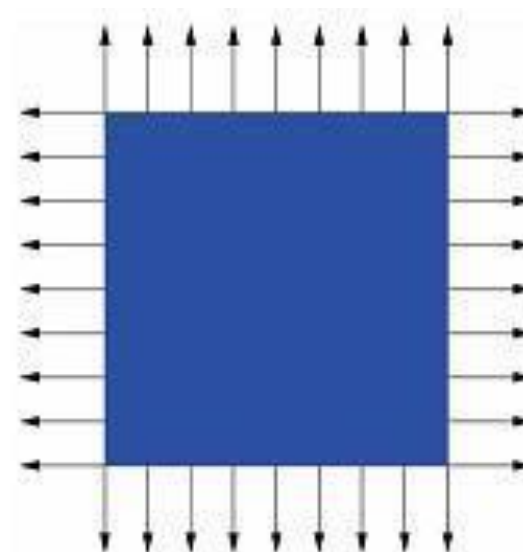
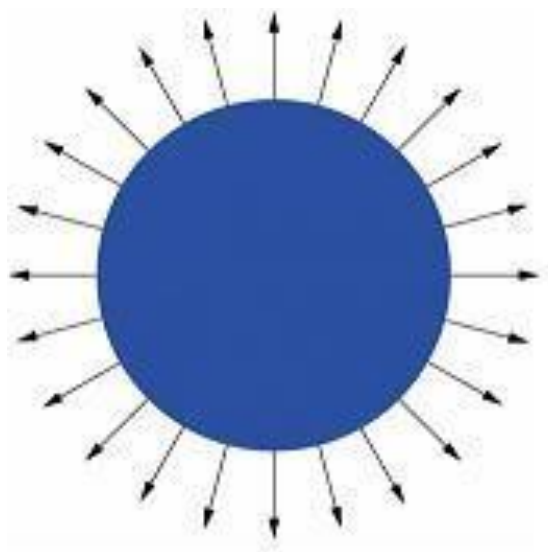


Transformation



Transformation of Normals (法向量变换)

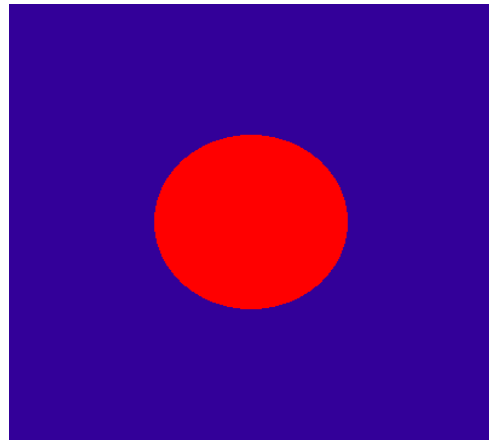
- Surface Normal: unit vector that is locally perpendicular to the surface. 局部垂直于曲面
- One of the most important geometric property of surfaces.



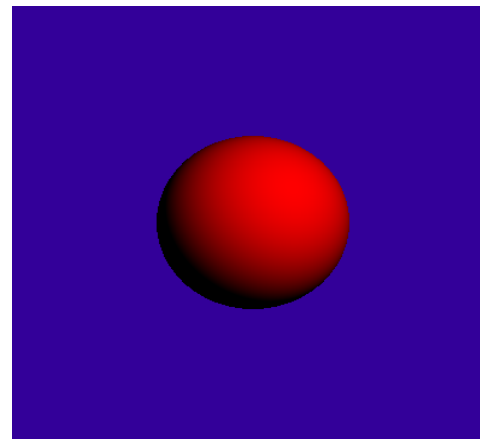
Why is the Normal important?

- It's used for shading — makes things look 3D!
- All kinds of lighting models utilized surface normal in the lighting computation

法向量是进行光照处理的必要输入，所有的光照模型都涉及到物体的法向量。只有知道了物体的法向量信息，才能绘制出具有三维立体感的图像。



object color only
只有物体颜色



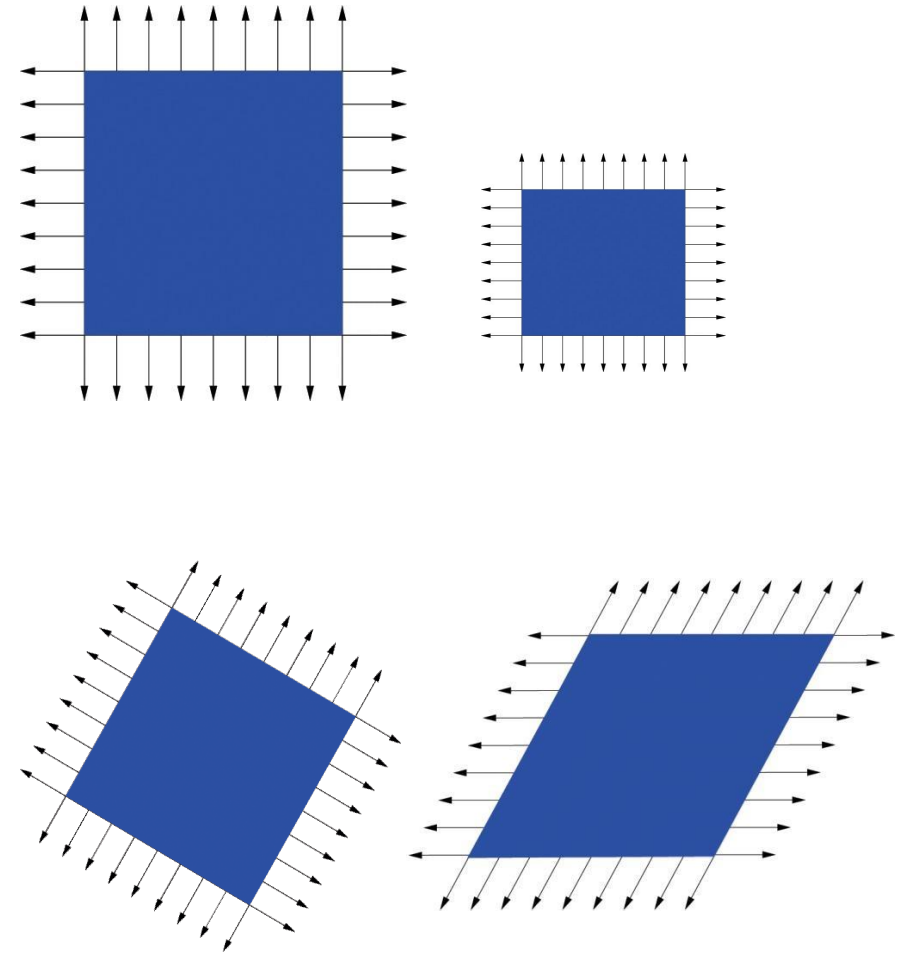
Diffuse Shading
漫反射着色

Transform Normal like Object?

- Could we transform the normal like objects? 在对物体进行变换时，法向量可以类似变换吗？
- We could find that translation, rotation and isotropic scale preserve the correct normal.

我们发现,对于相似变换,法向量可以和物体使用同样变换方程

- But **shear** does not (the right bottom image.)

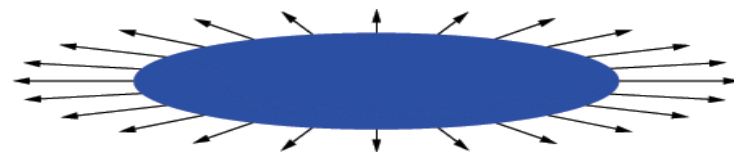
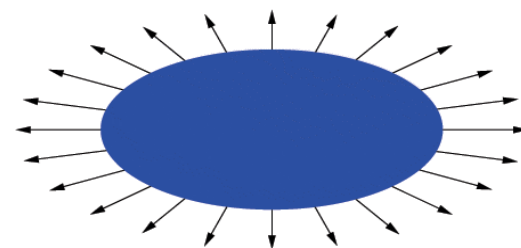
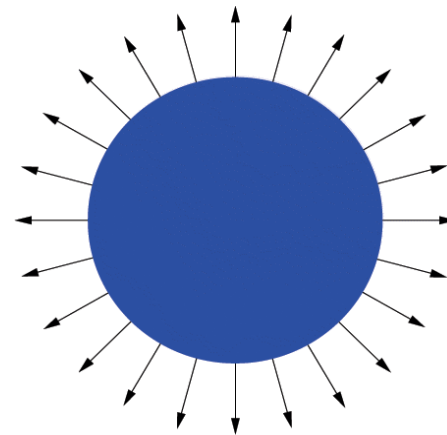


Transform Normal like Object?

- Another example
 - We scale an sphere (but **not isotropic scale**).
 - The transformed normal is also incorrect.

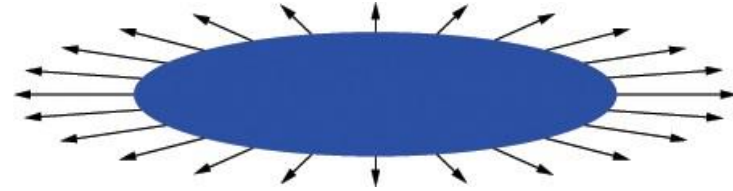
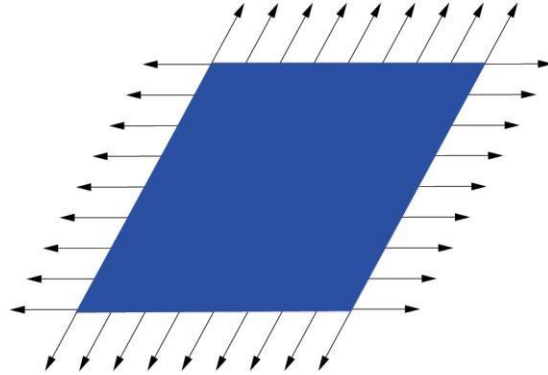
对右图的球做非均衡缩放。

使用与物体同样的变换方程用于变换法向量会出现错误结果。

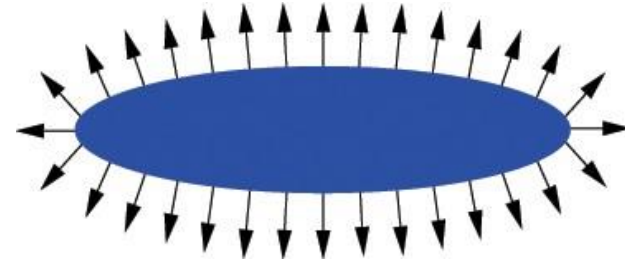
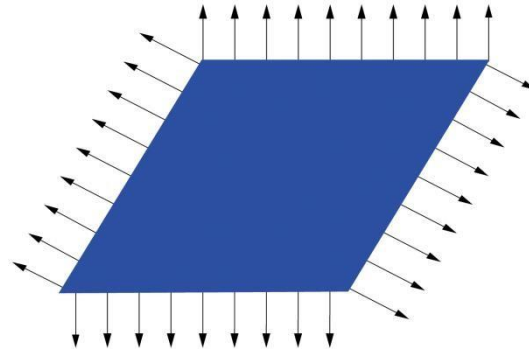


Transformation for shear and scale

Incorrect
Normal
Transformation



Correct
Normal
Transformation

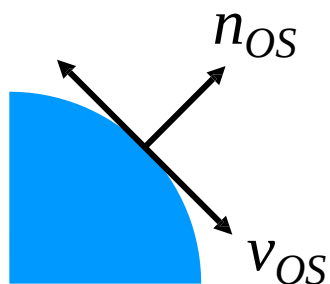


how could we transform normal correctly?

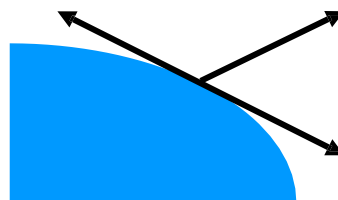
How to transform normal?

- transforming the *tangent plane* (切平面) to the normal, not the *normal vector directly*.

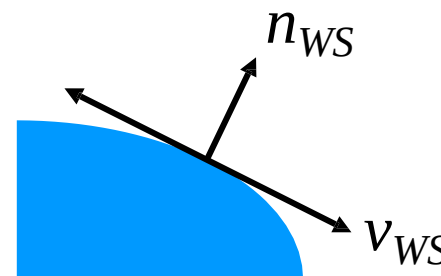
变换 切平面，再通过切平面计算法向量，而不是直接计算。



Original



Incorrect



Correct

Pick any vector v_{OS} in the tangent plane to transform
切平面上的任一向量 v_{OS} 变换后成为 v_{WS}

$$v_{WS} = \mathbf{M} v_{OS}$$

由切向量计算法向量: 切向量与法向量垂直

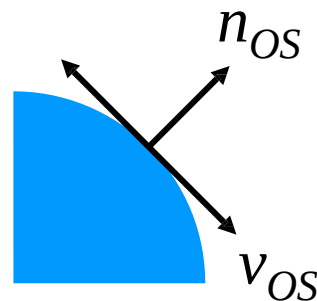
Transform tangent vector v_{OS} is perpendicular to normal n_{OS} :

$$\text{Dot product} \quad n_{OS}^T v_{OS} = 0$$

$$n_{OS}^T (\mathbf{M}^{-1} \mathbf{M}) v_{OS} = 0$$

$$(n_{OS}^T \mathbf{M}^{-1}) (\mathbf{M} v_{OS}) = 0$$

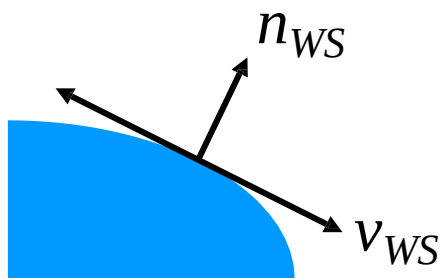
$$(n_{OS}^T \mathbf{M}^{-1}) v_{WS} = 0$$



v_{WS} is perpendicular to normal n_{WS} :

$$n_{WS}^T = n_{OS}^T (\mathbf{M}^{-1})$$

$$n_{WS} = (\mathbf{M}^{-1})^T n_{OS}$$



The matrix is the transpose of inverse of $\mathbf{M}$.

(法向量的变换矩阵是原变换矩阵的逆的转置)

图形软件标准Graphics Software Standard

□ 计算机图形标准是图形系统及其相关应用程序中各界面之间进行数据传送和通信的接口标准。

- **WHY?**

- 与设备无关的程序可移植性

OpenGL(Open Graphics Library)

- 跨平台开放式图形编程接口
- Developed by SGI美国硅图公司, independent with platform

OpenGL编程

- OpenGL的主要功能
- OpenGL的基本语法
- OpenGL的程序示例

OpenGL的主要功能

- 模型绘制
- 模型观察
- 颜色模式
- 光照应用
- 图像效果增强
- 位图和图像处理
- 纹理映射
- 实时动画
- 交互技术

OpenGL的基本语法——相关库

- **OpenGL核心库：**前缀是**gl**，包含115个最基本的命令函数，这些函数用于常规的、核心的图形处理，如建立各种集合模型，产生光照效果，进行反走样、投影变换等。
- **OpenGL实用程序库：****glu**，包含43个函数，这些函数通过调用核心库函数，实现一些较复杂的操作，如网格化、曲线曲面绘制等。
- **OpenGL实用程序工具包，****glut**，包含30多个函数，主要提供基于窗口的工具，后面程序中**均使用GLUT库**建立OpenGL程序运行框架。
- **OpenGL编程辅助库：****aux**
- **Windows专用库：****wgl**

OpenGL的基本语法——数据类型

表： OpenGL的数据结构

OpenGL数据类型	内部表示法	定义为C类型	C字面值后缀
GLbyte	8位整数	signed char	B
GLshort	16位整数	short	S
GLint, GLsizei	32位整数	long	L
GLfloat, GLclampf	32位浮点数	float	F
GLdouble, GLclampd	64位浮点数	double	D
GLubyte, GLboolean	8位无符号整数	unsigned char	Ub
GLushort	16位无符号整数	unsigned short	Us
GLuint, GLenum, GLbitfield	32位无符号整数	unsigned long	Ui

由于OpenGL具有平台无关性，而不同机器的数据类型所描述的范围可能不同，因此OpenGL定义了自己的数据类型。上述数据类型可以映射为常规的C数据类型。

OpenGL的基本语法——命名规则

- OpenGL的**基本函数**均以**gl**作为前缀，**基本常量**均以**GL_**开头。

例如基本常量：GL_TRIANGLES, GL_POLYGON。

- OpenGL**函数**都遵循一个命名约定，即采用以下格式：

<库前缀><根命令><可选的参数个数><可选的参数类型>

例如函数**glColor3f**(...), gl表示这个函数来自库gl.h, color表示该函数用于颜色设定, 3f表示这个函数采用了3个浮点数参数。

OpenGL程序实例

OpenGL绘图的一般过程，通常包括以下步骤：

- 头文件包含
- 使用GLUT库实现窗口管理
- 利用OpenGL绘图

OpenGL程序实例——头文件包含

- 使用OpenGL的核心库
 - `#include <windows.h>`
 - `#include <GL/gl.h>`
 - `#include <GL/glu.h>`
- 使用窗口管理，`glut.h`自动包含了`gl.h`和`glu.h`。
 - `#include <GL/glut.h>`
- C程序所需的头文件
 - `#include <stdio.h>`
 - `#include <stdlib.h>`
 - `#include <math.h>`

OpenGL程序实例——窗口管理

- 初始化（ **glutInit** ） 初始化 GLUT 库
- 设定窗口的显示模式（ **glutInitDisplayMode** ）
- 指定窗口的位置和大小（ **glutInitWindowPosition** 和 **glutInitWindowSize** ）
- 创建窗口（ **glutCreateWindow** ）
- 指定窗口的显示内容函数（ **glutDisplayFunc** ） 一般是用于绘图的函数
- 运行窗口框架（ **glutMainLoop** ） 进入GLUT事件处理循环, 开始绘制

如：函数`glutInitWindowSize(400,300)` 和 `glutInitWindowPosition(100,120)` 定义了一个距离屏幕左边界100像素，距离屏幕上边界120像素，宽度为400像素，高度为300像素的窗口。

OpenGL程序实例——绘制图形

- 指定窗口背景色（`glClearColor`）
- 颜色管理，使用RGB颜色模型。一种颜色用红、绿、蓝三种颜色成分混合而成，每种颜色成分使用0.0到1.0之间的任意有效浮点数来表示颜色值。

如：函数`glClearColor(1.0f, 1.0f, 1.0f)`指定窗口的背景为白色。

OpenGL程序实例——绘制图形

表：OpenGL的一些常用混合色：都被归一化在（0，1）之间的值。

0.0表示不使用该种颜色，而1.0表示将该种颜色用到最多，混合的效果。

混合色	红色成分（R）	绿色成分（G）	蓝色成分（B）
黑	0.0	0.0	0.0
红	1.0	0.0	0.0
绿	0.0	1.0	0.0
黄	1.0	1.0	0.0
蓝	0.0	0.0	1.0
紫	1.0	0.0	1.0
青	0.0	1.0	1.0
深灰	0.25	0.25	0.25
浅灰	0.75	0.75	0.75
棕	0.60	0.40	0.12
南瓜橙	0.98	0.625	0.12
粉红	0.98	0.04	0.70
紫红	0.60	0.40	0.70
白	1.0	1.0	1.0

OpenGL程序实例——绘制图形

- 指定窗口的背景色后，在每次绘制图形前需要用到函数**glClear**，因为**glClearColor**只是设置了背景颜色，并未实际进行清除操作。
glClear函数用**glClearColor**函数中指定的值设定颜色缓冲区中的值，即将窗口中每个像素的颜色设为背景色。
- 清除颜色缓冲区：
glClear(GL_COLOR_BUFFER_BIT);

OpenGL程序实例——绘制图形

- 在绘制完图形之后，一般使用`glflush`函数来结束绘图，通常在每一帧的绘制末尾调用。
- 该函数让前面的OpenGL绘制命令开始执行，强制执行OpenGL命令。
- `glflush( );`

OpenGL程序实例——绘制图形

- 绘制基本几何元素：顶点，边，多边形
- **图元**由下面语句定义：

```
glBegin( primType );  
glEnd();
```

 - *primType* 决定顶点如何组合成图元
- OpenGL中：**glBegin**函数标志几何元素绘制开始，**glEnd**函数表示集合元素绘制结束。通过设置glBegin()的参数来指定绘制何种几何元素。注意，一个glBegin函数必须搭配一个glEnd函数。
- OpenGL 通过使用**glVertex**函数来定义**顶点**， glVertex函数必须出现在**glBegin**和**glEnd**之间。
- 函数void glBegin(Glenum mode)中mode值如表所示：

OpenGL程序实例——绘制图形

mode:创建元素的类型

mode 值	几何图元类型
GL_POINTS	一系列独立的顶点
GL_LINES	每两点构成一条线段
GL_POLYGON	简单、凸多边形的边界
GL_TRIANGLES	每三点连成一个三角形
GL_QUADS	每四点连成一个四边形
GL_LINE_STRIP	顶点相连，成为一系列线段
GL_LINE_LOOP	顶点相连，成为一系列线段，最后一点与第一点相连
GL_TRIANGLE_STRIP	相连的三角形带
GL_TRIANGLE_FAN	相连的三角形扇形
GL_QUAD_STRIP	相连的四边形带

点的绘制

- 点的绘制

```
glBegin(GL_POINTS);  
    glVertex3f(0.0f, 0.0f, 0.0f);  
    glVertex3f(10.0f, 0.0f, 0.0f);  
glEnd();
```

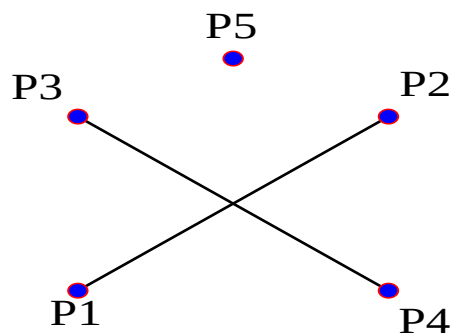
- 点的属性（大小）

```
void glPointSize(GLfloat size)
```

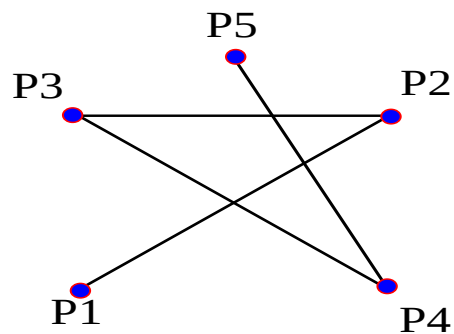
大小的度量单位为屏幕中的像素，默认值为1.0

直线的绘制

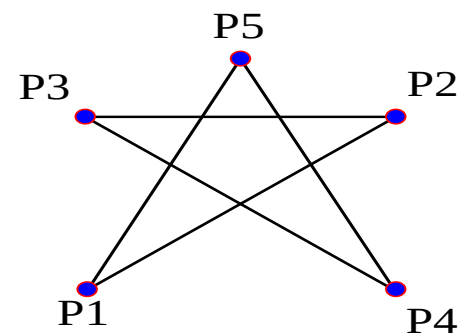
- 直线的绘制模式
 - GL_LINES
 - GL_LINE_STRIP
 - GL_LINE_LOOP



(a) GL_LINES画线模式



(b) GL_LINE_LOOP画线模式
图5-57 OpenGL画线模式



(c) GL_LINE_STRIP画线模式

直线的绘制

- 直线的属性

- 线宽

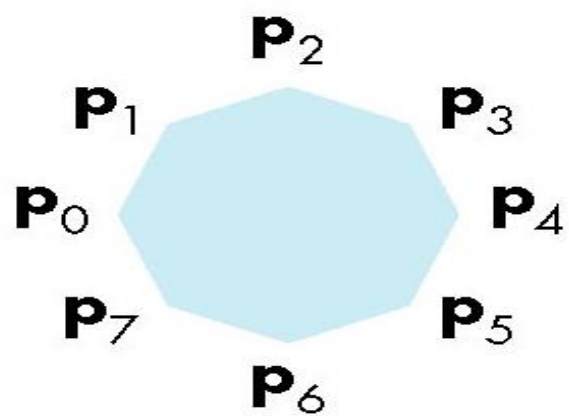
`void glLineWidth(GLfloat width)`

单位为像素，默认值为1.0

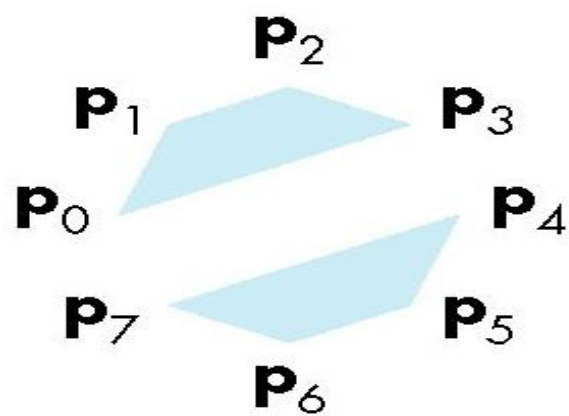
多边形面的绘制

- 三角形面的绘制
 - GL_TRIANGLES
 - GL_TRIANGLE_STRIP
 - GL_TRIANGLE_FAN
- 四边形面的绘制
 - GL_QUADS
 - GL_QUADS_STRIP
- 多边形面的绘制 (GL_POLYGON)

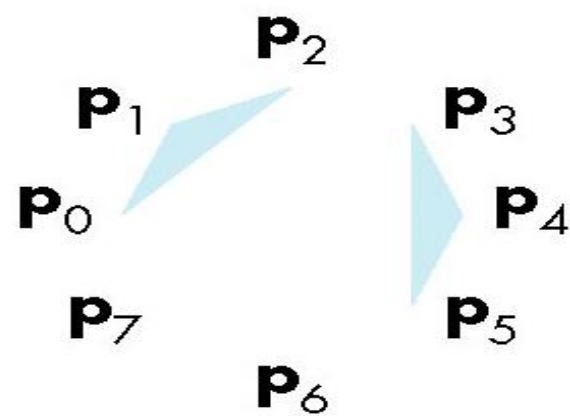
多边形面的绘制



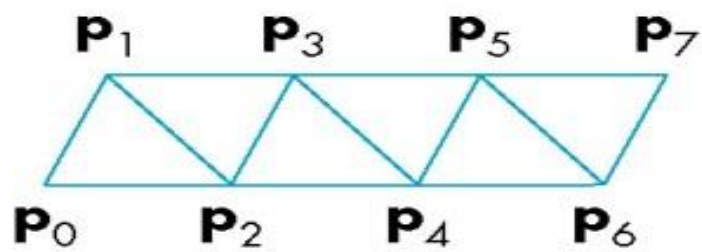
GL_POLYGON



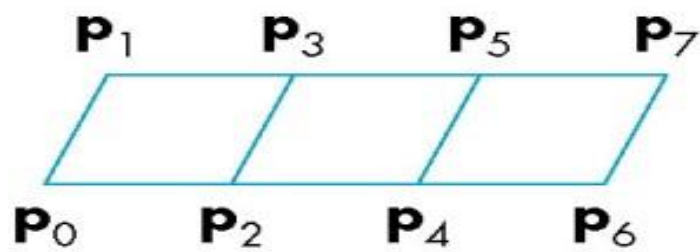
GL_QUADS



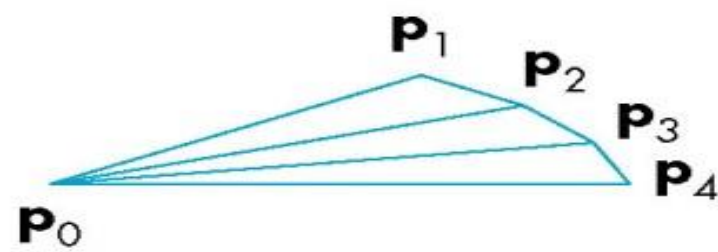
GL_TRIANGLES



GL_TRIANGLE_STRIP



GL_QUAD_STRIP



GL_TRIANGLE_FAN

矩形

绘制矩形函数：

glRectf(50.0f, 100.0f, 150.0f, 50.0f);

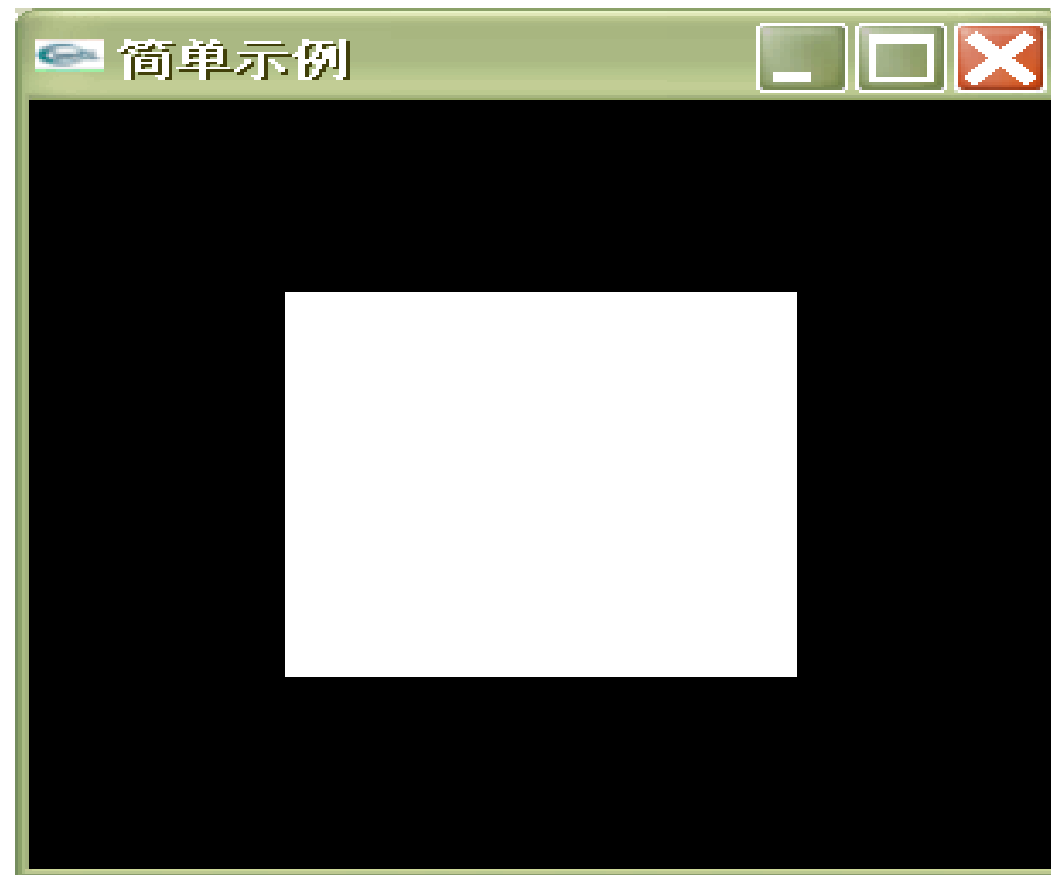
//绘制一个左上角点在(50.0,100.0),右下角点在(150.0,50.0)的矩形。

程序示例

示例

一个简单程序

- 在黑色背景上画一个白色矩形

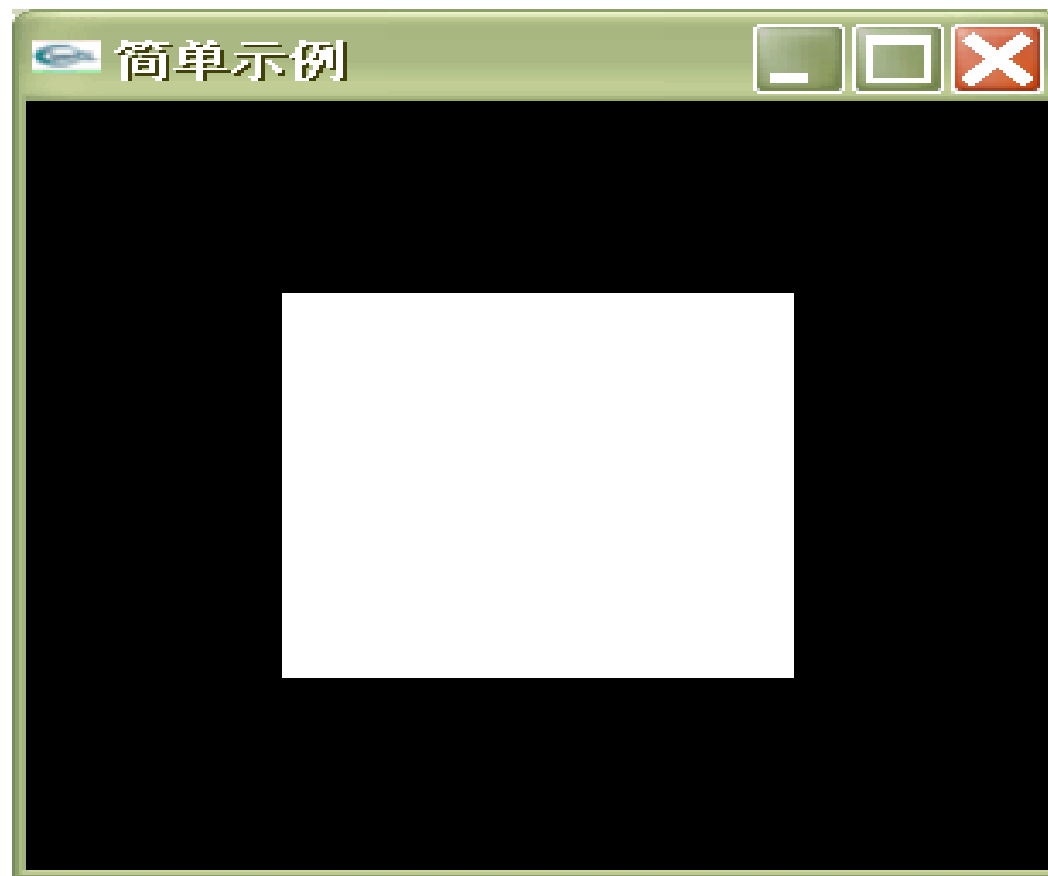


simple.c

```
#include <GL/glut.h>
void mydisplay() { // display函数定义了所要绘制的矩形
    glClear(GL_COLOR_BUFFER_BIT); //清空颜色缓冲区,实际完成清除窗口的任务
    glBegin(GL_POLYGON);
        glVertex2d(-0.5, -0.5); glVertex2d(-0.5, 0.5);
        glVertex2d(0.5, 0.5);    glVertex2d(0.5, -0.5);
    glEnd();
    glFlush(); //保证了绘图命令将实际执行, 强制执行OpenGL命令
}
int main(int argc, char* argv[]) { //main函数负责OpenGL的初始化
    glutInit(&argc, argv); //函数glutInit接收来自main函数的参数, 初始化GLUT
    glutCreateWindow("简单示例"); //以"简单示例"为标题创建一个默认尺寸为300x300像素显示在屏幕默认位置（左上角）的窗口, 在调用glutMainLoop之前, 这个窗口并没有显示。
    glutDisplayFunc(mydisplay); //注册显示回调函数。把mydisplay函数注册为一个显示回调函数。
    //当窗口需要重绘时, mydisplay函数就会被调用。
    glutMainLoop(); //使程序进入GLUT事件处理循环。启动程序, 所有已创建的窗口会在这时显示, 对这些窗口的渲染也开始生效。事件处理循环开始启动, 已注册的显示回调函数被触发。main函数最后一条语句。
}
```

- 修改刚刚程序
 - 改变默认值
 - 引进**程序的标准结构**

在黑色背景上画一个白色矩形



程序结构

- 绝大多数OpenGL程序具有类似的结构，包含下述函数
 - **main()**:
 - 定义回调函数
 - 打开一个或多个具有指定属性的窗口
 - 进入事件循环（最后一条可执行语句）
 - **init()**: 设置OpenGL状态变量
 - 视图
 - 属性
- 显示回调函数: 用于绘图

程序结构

```
#include <GL/glut.h>
```

← 这自动包含了gl.h,glu.h

```
void init() {...} // 设置OpenGL状态  
void display() {...} // 显示回调函数  
int main(int argc, char** argv)  
{
```

```
    glutInit(&argc,argv);
```

```
    ...
```

```
    //创建并初始化GLUT窗口;
```

```
    ...
```

```
    glutDisplayFunc(display);
```

← 显示回调函数

```
    ...
```

```
    注册其他回调函数;
```

```
    ...
```

```
    init();
```

← 设置OpenGL状态

```
    glutMainLoop();
```

← 进入事件循环

```
}
```

Main函数

```
#include <GL/glut.h>
```

← 这自动包含了gl.h

```
int main(int argc, char** argv)
{
```

```
    glutInit(&argc,argv); //初始化GLUT
```

```
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB); //设置窗口的显示属性(单缓冲, RGB颜色)
```

← 定义窗口属性

```
    glutInitWindowSize(300,300);
```

```
    glutInitWindowPosition(0,0); //窗口左上角相对于屏幕左上角的位置
```

```
    glutCreateWindow("简单示例");
```

```
    glutDisplayFunc(mydisplay);
```

← 显示回调函数

```
    init();
```

← 设置OpenGL状态

```
    glutMainLoop();
```

← 进入事件循环

```
}
```

Init函数

```
void init()  
{
```

```
    glClearColor(0.0, 0.0, 0.0, 1.0);
```

清除色（背景色）为黑色

```
    glColor3f(1.0, 1.0, 1.0);
```

设置当前的绘图颜色:矩形填充以白色

```
    glMatrixMode(GL_PROJECTION); //设置矩阵类型是投影矩阵
```

```
    glLoadIdentity(); //单位矩阵
```

```
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
```

定义正交投影视景体(裁剪窗口)

```
}
```

Mydisplay函数

```
void mydisplay()  
{  
    glClear(GL_COLOR_BUFFER_BIT); // 清空缓冲区  
    glBegin(GL_POLYGON); // 绘制图形  
        glVertex2d(-0.5,-0.5);  
        glVertex2d(-0.5,0.5);  
        glVertex2d(0.5,0.5);  
        glVertex2d(0.5,-0.5);  
    glEnd();  
    glFlush(); // 强制执行OpenGL命令  
}
```



```

/** simple2.c
 * This program draws a white rectangle on a black background.*/
#include <GL/glut.h>      /* glut.h includes gl.h and glu.h*/
void display(void)
{
    /* clear window */
    glClear(GL_COLOR_BUFFER_BIT);    // 清空缓冲区
    /* draw unit square polygon */
    glBegin(GL_POLYGON);            // 绘制图形
        glVertex2f(-0.5, -0.5);
        glVertex2f(-0.5, 0.5);
        glVertex2f(0.5, 0.5);
        glVertex2f(0.5, -0.5);
    glEnd();

    /* flush GL buffers */
    glFlush();        // 强制执行OpenGL命令
}
void init()
{
    /* set clear color to black */
    glClearColor (0.0, 0.0, 0.0, 0.0);    //设置窗口背景颜色
    /* set fill color to white */
    glColor3f(1.0, 1.0, 1.0);            //设置当前的绘图颜色
    /* set up standard orthogonal view with clipping */
    /* box as cube of side 2 centered at origin */
    /* This is default view and these statement could be removed */
    glMatrixMode (GL_PROJECTION); //设置矩阵类型是投影矩阵
    glLoadIdentity (); //单位矩阵
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0); //定义正交投影视景体(裁剪窗口)
}
int main(int argc, char** argv)
{
    /* Initialize mode and open a window in upper left corner of screen */
    /* Window title is name of program (arg[0]) */
    glutInit(&argc,argv);        //初始化GLUT
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB); //设置窗口的显示属性(单缓冲，RGB颜色)
    glutInitWindowSize(300,300); // 设置窗口的初始宽度和高度，单位为像素，缺省300x300
    glutInitWindowPosition(0,0); // 窗口左上角相对于屏幕左上角的位置，单位为像素，缺省(0,0)
    glutCreateWindow("simple"); //创建窗口
    glutDisplayFunc(display); //注册显示回调函数：窗口需要重绘时调用display
    init();        //设置OpenGL状态
    glutMainLoop(); //事件处理循环：main函数最后一条语句
}

```

完整程序

Thanks!